PXL – Digital
421280 Software Analysis

Model Based Documentation of Requirements – State Transition diagram

Week 11 – period 01

Luc Doumen

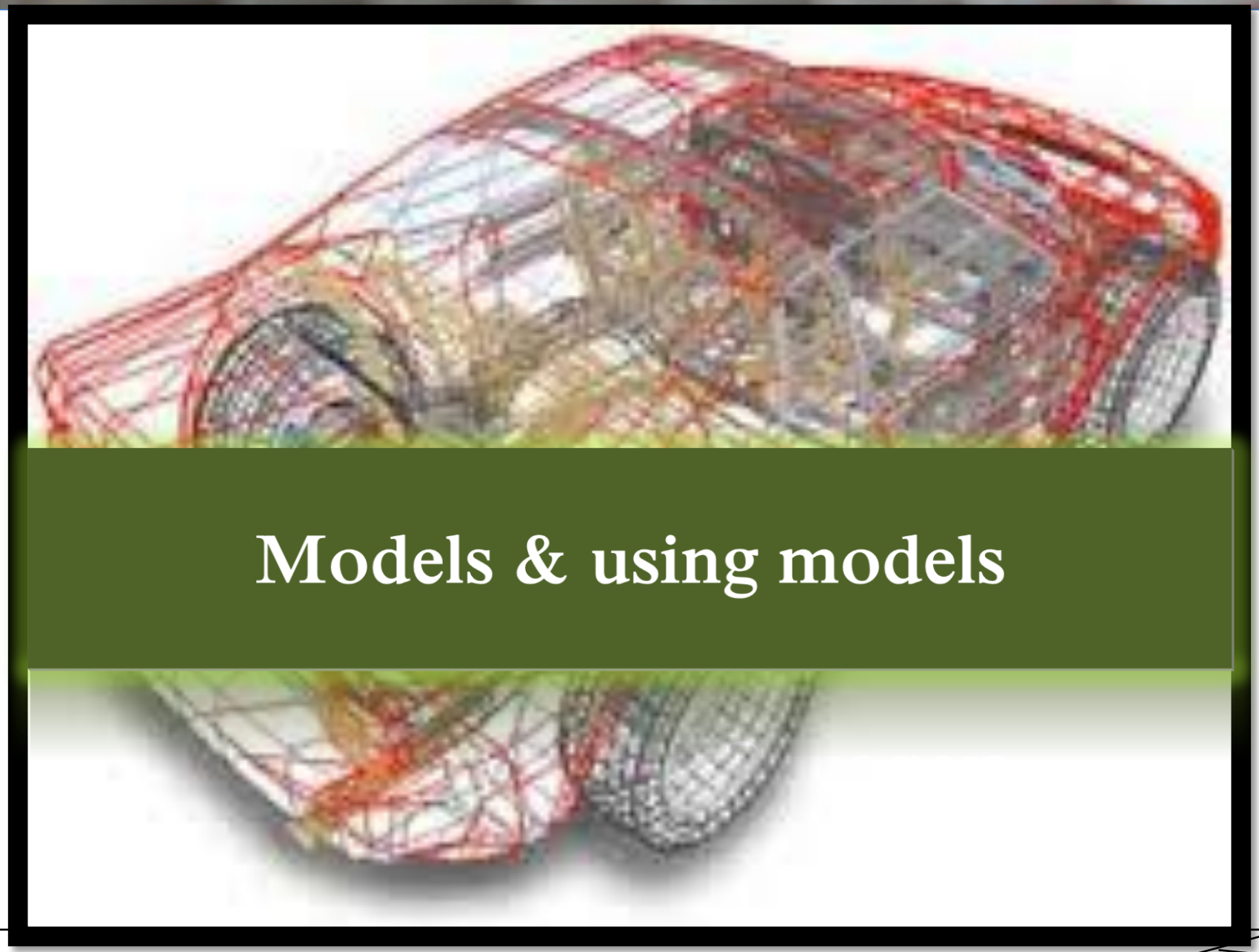
Nathalie Fuchs

DE HOGESCHOOL MET HET NETWERK

Elfde-Liniestraat 24, 3500 Hasselt, www.pxl.be

Content

- Models & using models
- Topic overview
  - Goal models
  - Use models (system use cases)
  - 3 perspectives on requirements
    - Data/structural: ERM, Class models
    - Functional: Data Flow Diagram, Activity diagram
    - Behavioral: State charts
  - Sequence diagram
- Key learning points
- Questions & answers



Models & using models

Topic Overview

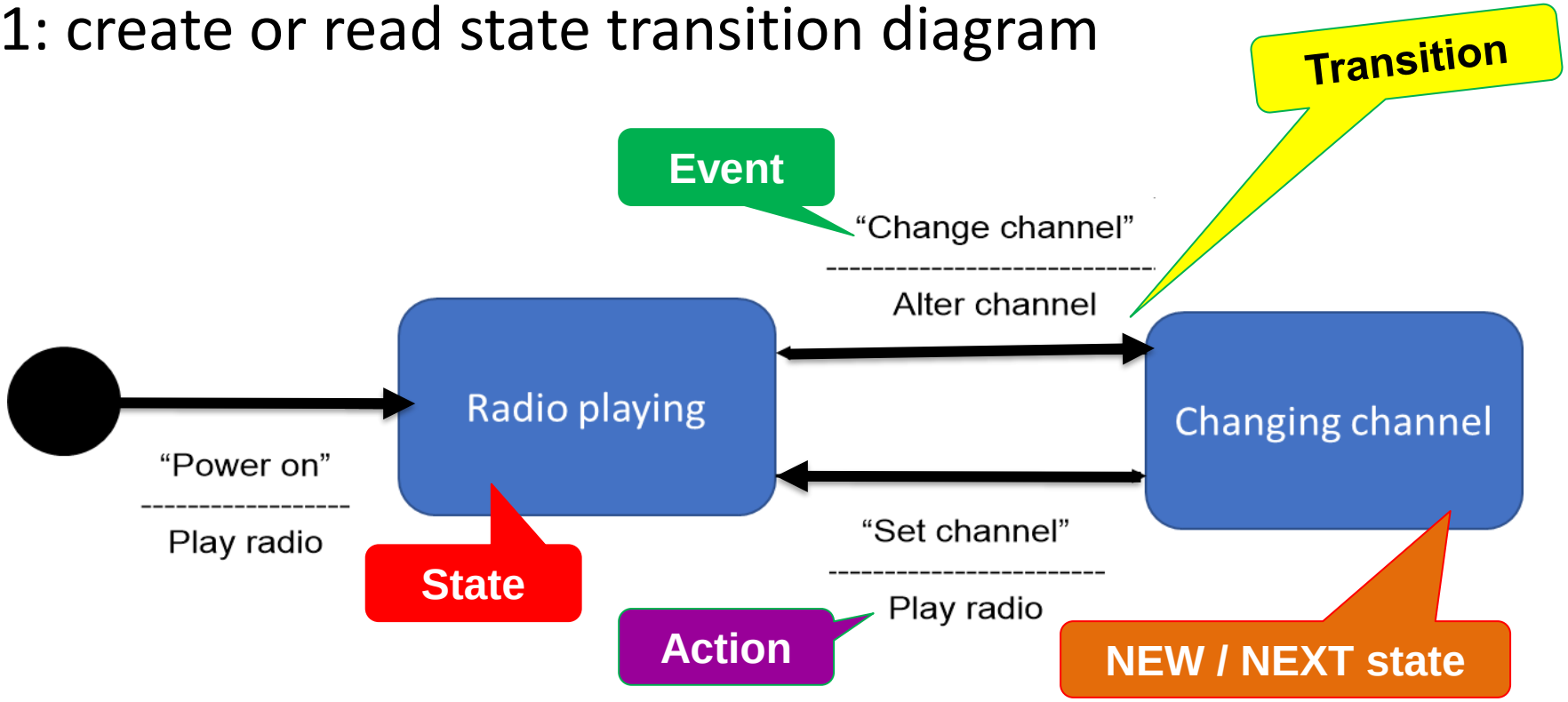
- Goal models
- System use case models (+ descriptions)
- **Three perspectives on requirements**
  - Data perspective
    - ERM, Class models (UML)
  - Functional perspective
    - Data flow diagrams, Activity diagrams (UML)
  - **Behavioral perspective**
    - State charts
- Sequence diagrams

Behavioral Requirements Models

- For modeling the dynamic behavior of a system
- Focus is on system states and events that can change that state
- Behavioral models
- UML State diagrams
  - Based on principles of finite state machines
  - Modeling elements: state, start- and end states, state transition, concurrency

State Transition Diagram (STD) – steps (1)

- Step 1: create or read state transition diagram



- **<u>Remark</u>**: a transition is only executed when its guard (event) is true

State Transition Diagram (STD) – steps (1)

- Step 2: create state transition table

Events

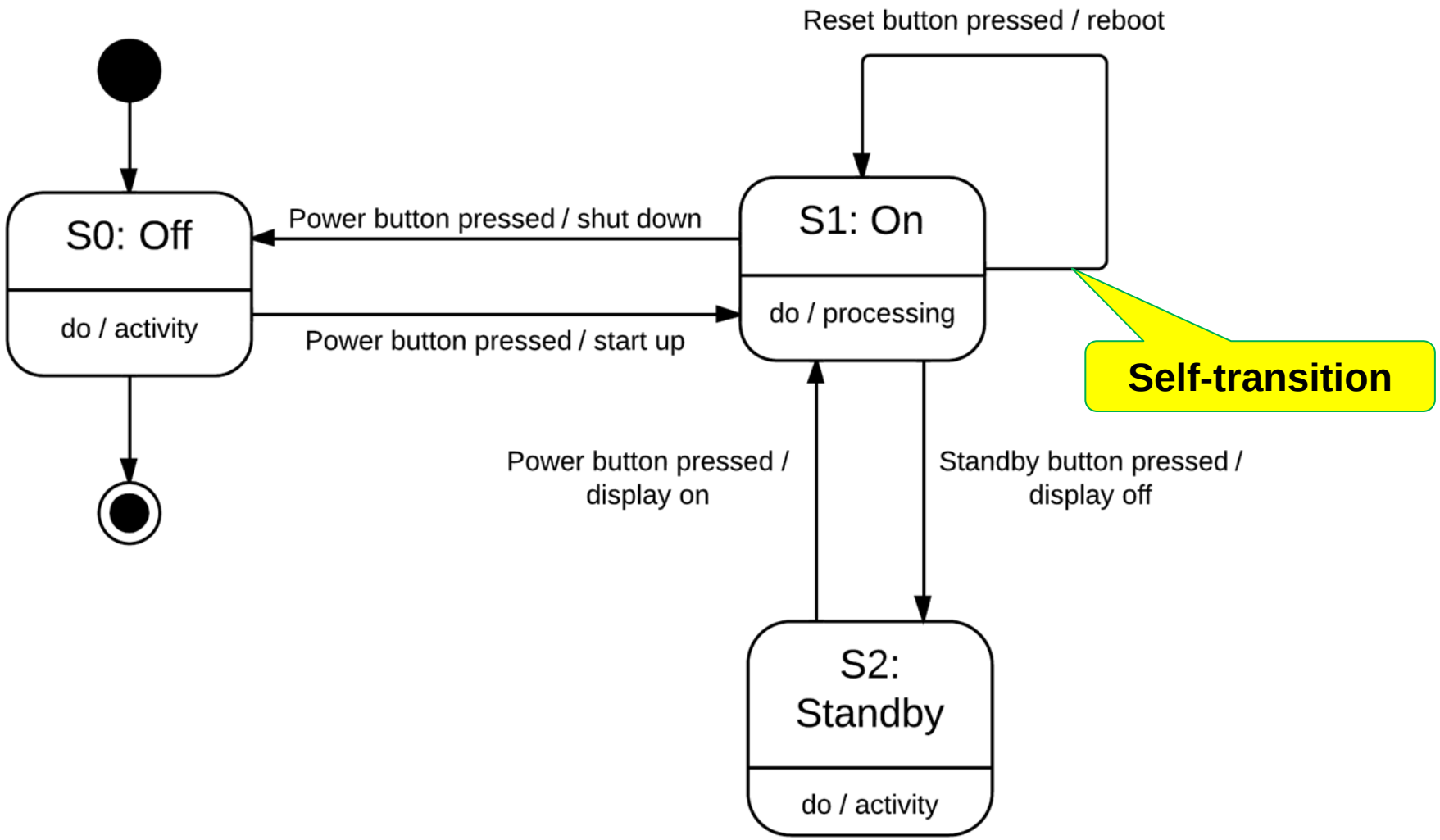
EVENT STATE	Power on	Change channel	Set channel
NULL	**A:** Play radio **S_{new}:** Radio playing	n.a.	n.a.
Radio playing	n.a.	**A:** Alter channel **S_{new}:** Channel changing	n.a.
Channel changing	n.a.	n.a.	**A:** Play radio **S_{new}:** Radio playing

States

Action

NEW / NEXT state

STD – example laptop



STD - how to draw?

- In 5 steps
  - Define states
  - Describe states (for others to understand)
  - Draw transitions
  - Define events / guard conditions
  - Define actions
  - Create state transition table

STD - pros and cons

- Describes the behaviour of a system
- All possible states a system can get into
- State changes as result of events
- Actions are short effects of transitions

☺ Suitable for modelling interfaces

☺ Suitable for simulation and testing

☹ May be difficult to read for outsiders

STD - Case - Website

- A website has a homepage, by choosing the button contact ‘ contact information’ is provided.
- By choosing the button ‘services’ an overview is provided of the services.
- The ‘services’ page also has a hyperlink to the contact information page.
- The home button will bring you back to the homepage.

- Draw the <u>state transition diagram</u>
- Create the <u>state transition table</u>
- You are allowed to ask questions!

STD - Case - Digital pet program

- You're creating a digital pet program. What happens to the pet when he receives different stimuli is determined by the state, he's in, so you decide to model the digital pet with a <u>state diagram</u>.
- The behavior of the digital pet program is as follows:
  - When the pet is turned on, it starts out happy
  - If the pet is happy and receives punishment, then he becomes sad
  - If the pet is sad and receives praise, it becomes happy
  - If the pet is sad and receives punishment, it is heart-broken
  - Create also the <u>state transition table</u>

STD - Case - Fuel pump

- Model the behavior of a fuel pump controller
  - User can buy fuel after inserting a credit card, which is read and validated by the controller
  - Then the user takes the hose out of the holster, and pushes the nozzle trigger, to fuel his car
  - When the nozzle is off, the fuel flow is stopped and the price is charged on the credit card
  - If invalid card or timeout the system returns to the initial waiting state

- Draw the <u>state transition diagram</u>
- Create the <u>state transition table</u>

STD - Case – VR glasses

As I slip on the VR glasses, the world around me fades into standby mode. The glasses are powered off, and there's a quiet anticipation before I power them on. With a tap, the glasses come to life, transitioning to a boot-up sequence. I wait as the system initializes, signaling the start of my VR journey.

Soon, I'm greeted by the main menu, a gateway to various experiences. I navigate through options—games, applications, settings—using hand gestures or controllers. Selecting an application triggers a transition, plunging me into a different world altogether.

The chosen game or application runs seamlessly, wrapping me in its immersive environment. I explore, interact, and experience as if I'm truly there. Time flies until I decide to pause or exit. With a simple gesture or button press, I return to the main menu, back to reality momentarily.

Sometimes, I delve into the settings, tweaking preferences for display, audio, or connectivity. The glasses adapt to my liking, ensuring a personalized experience. I might recalibrate sensors or adjust the interpupillary distance, fine-tuning the immersion.

Eventually, it's time to power off. The transition back to standby mode is smooth, leaving me in the quiet of the initial state. But occasionally, hiccups occur—a warning flashes due to low battery or a technical glitch. These moments prompt a transition to troubleshoot or require a restart, bringing me back to the start.

In this world of virtual experiences, these transitions—powering on, selecting, immersing, and returning—are my guideposts. They map the journey within this digital realm, ensuring that each time I wear these glasses, the transitions flow seamlessly, enveloping me in worlds beyond imagination.

- Draw the state transition diagram and create the state transition table

STD - Case – Laundry machine

As I load the laundry into the washing machine, it rests in its initial state—quiet and idle. With a push of a button, the machine powers on, initiating the cycle of washing. There's a gentle hum as it transitions into action.

From the initial state, it progresses into various stages—filling with water, agitating, rinsing, and spinning. Each stage represents a transition, taking the laundry from soiled to spotless. It's fascinating to watch the machine transition seamlessly between these states, orchestrating the cleaning process.

During the wash cycle, I might intervene, adding detergent or adjusting settings, which serves as smaller transitions within the larger washing process. These adjustments personalize the experience, tailoring the wash to different fabric types or soil levels.

Once the cycle completes, the machine transitions to a paused state, waiting for me to unload the clean clothes. Opening the lid or door marks the end of the washing cycle, transitioning back to the initial state, ready for the next load.

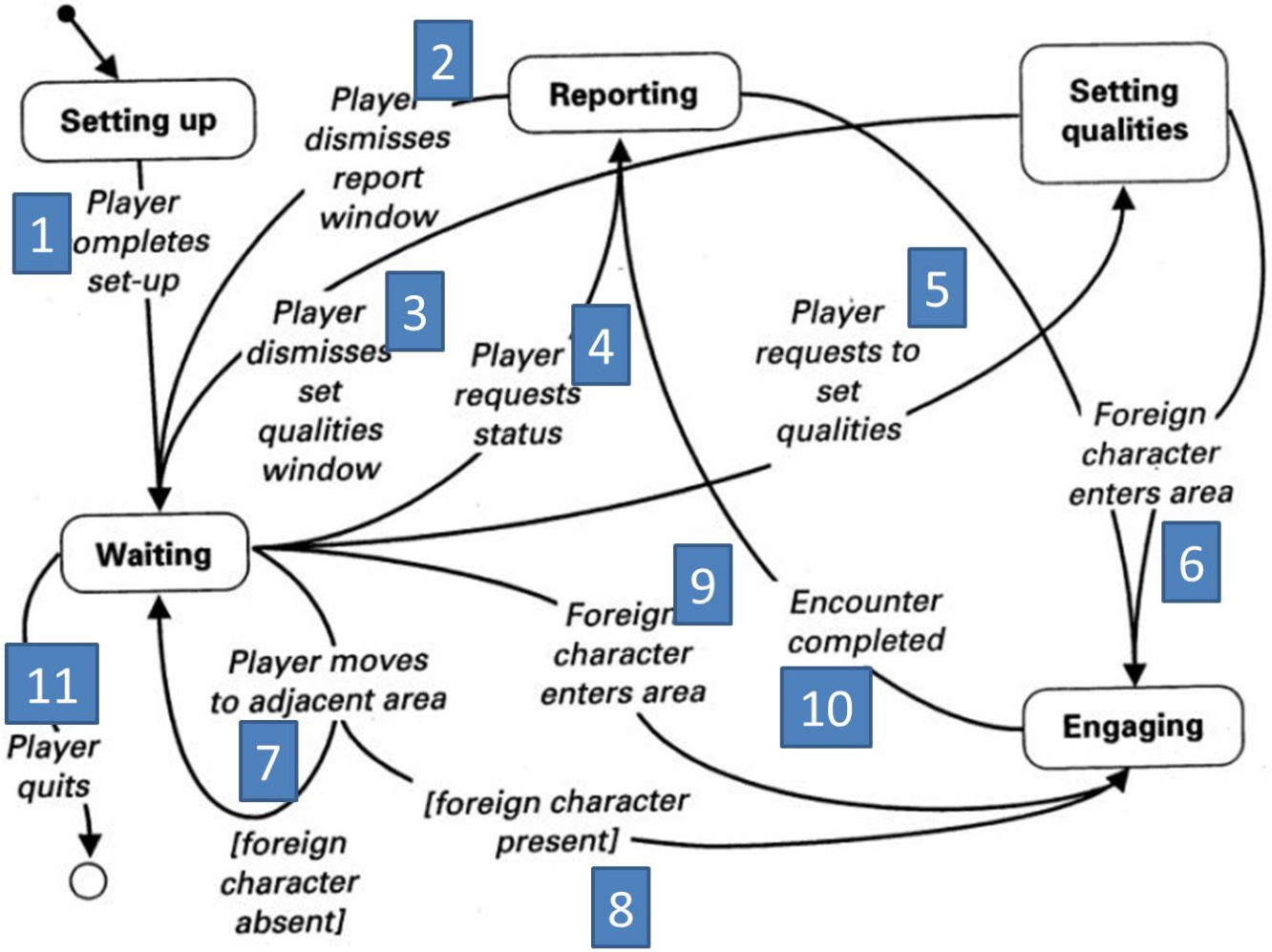
Occasionally, there might be interruptions—a warning signal due to unbalanced loads or an error in the cycle. These moments trigger transitions that require attention, sometimes necessitating a manual intervention to resolve the issue before resuming the washing process.

In the realm of household chores, these transitions within the washing machine represent a familiar rhythm. They mark the journey from dirty to clean, orchestrating the symphony of water, soap, and motion that transforms laundry with each cycle.

- Draw the state transition diagram and create the state transition table

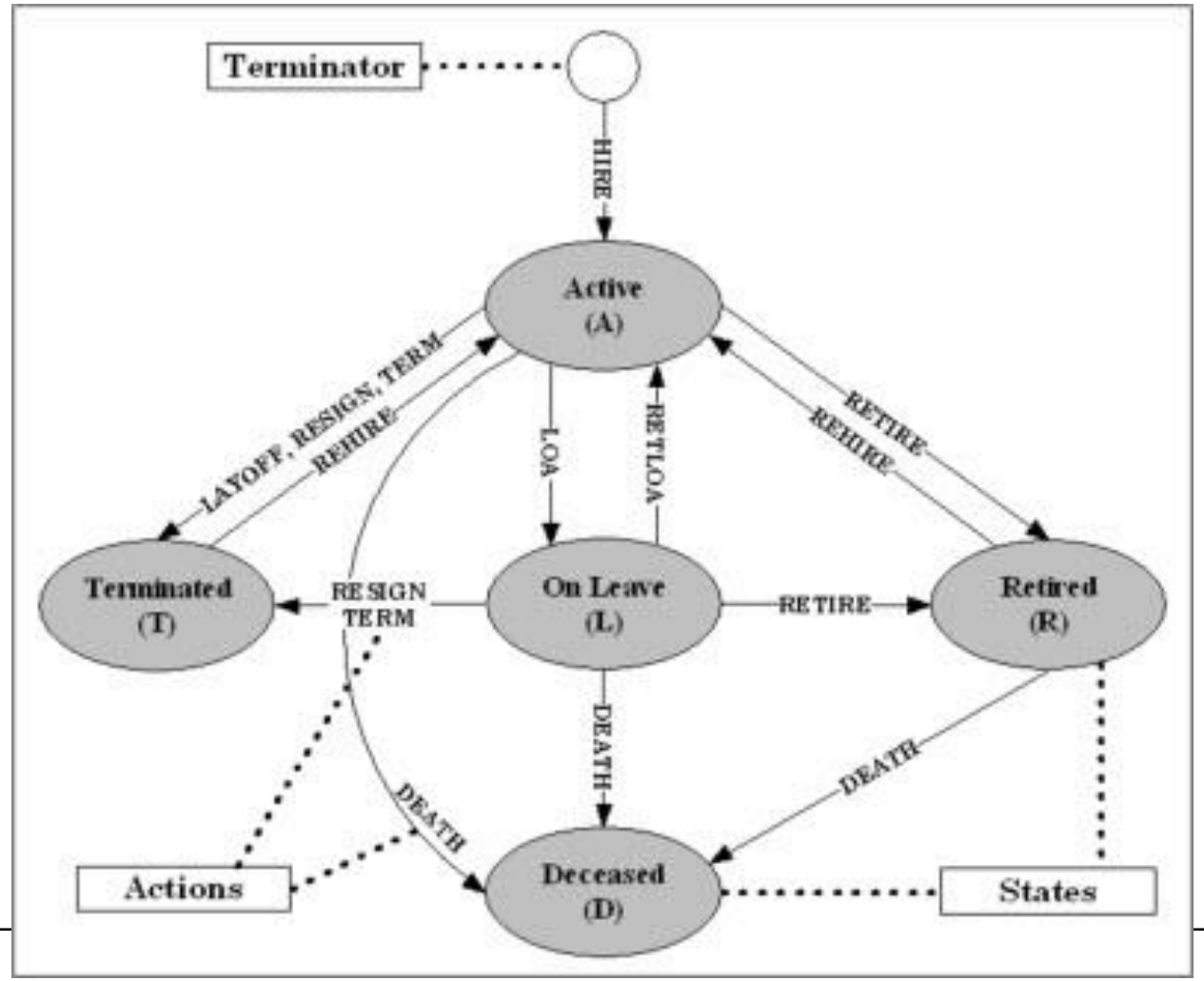
STD - Case - Game

- Create the <u>state transition table</u>



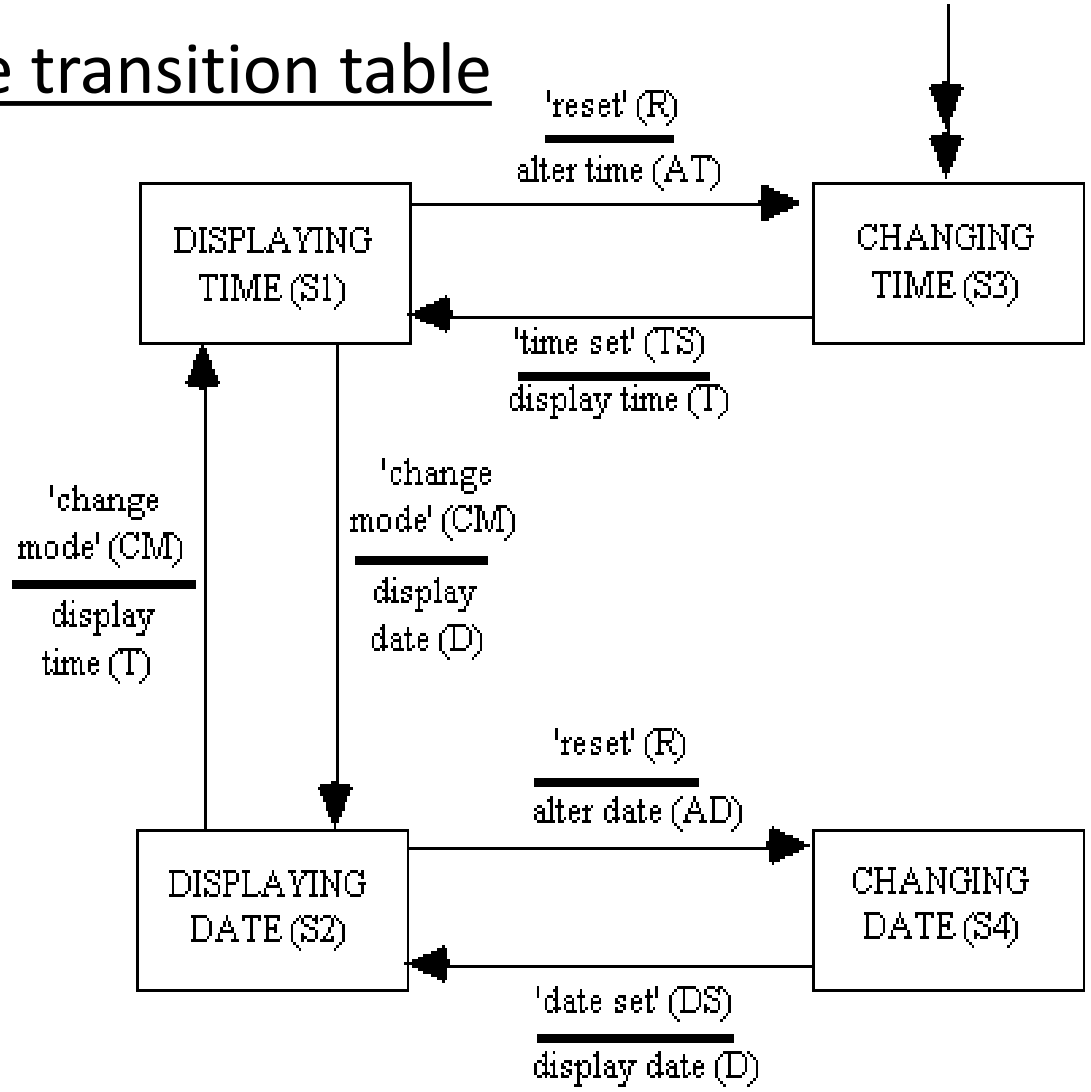
STD - Case - Person's status

- Create the <u>state transition table</u>



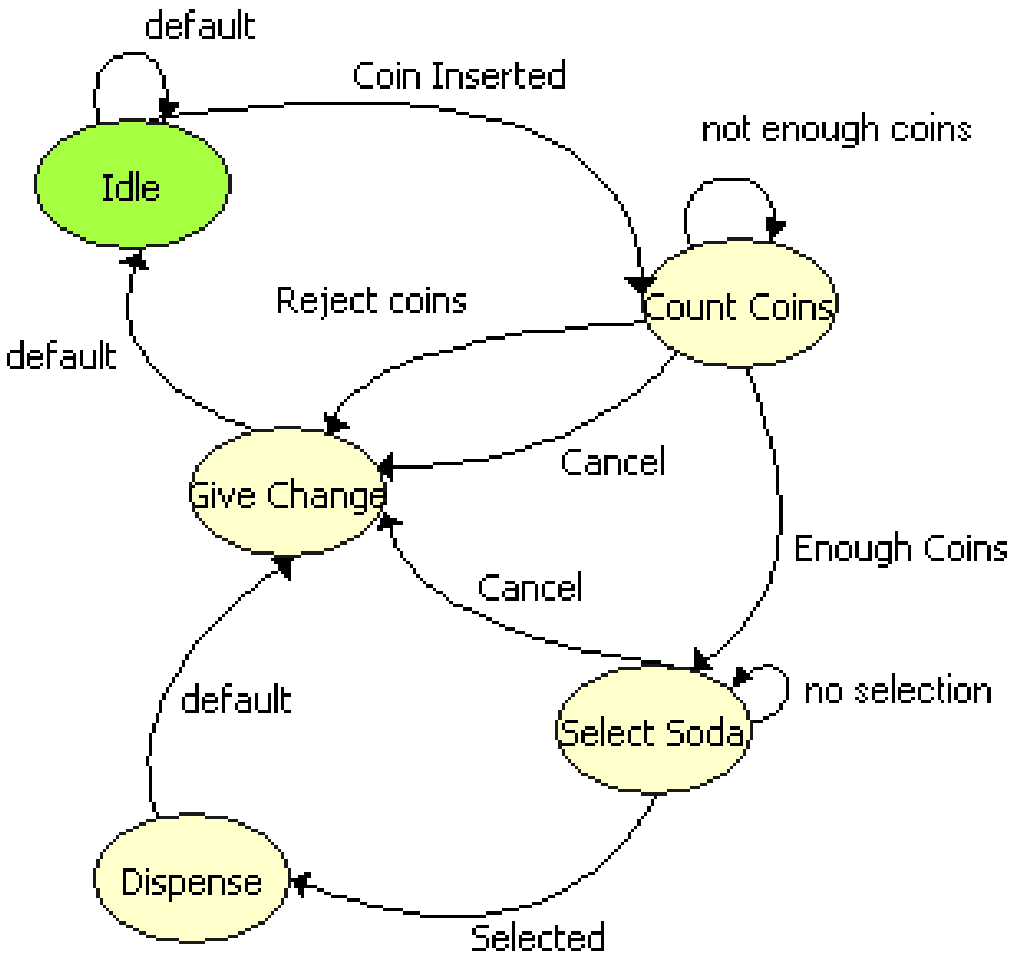
STD - Case - Clock

- Create the <u>state transition table</u>



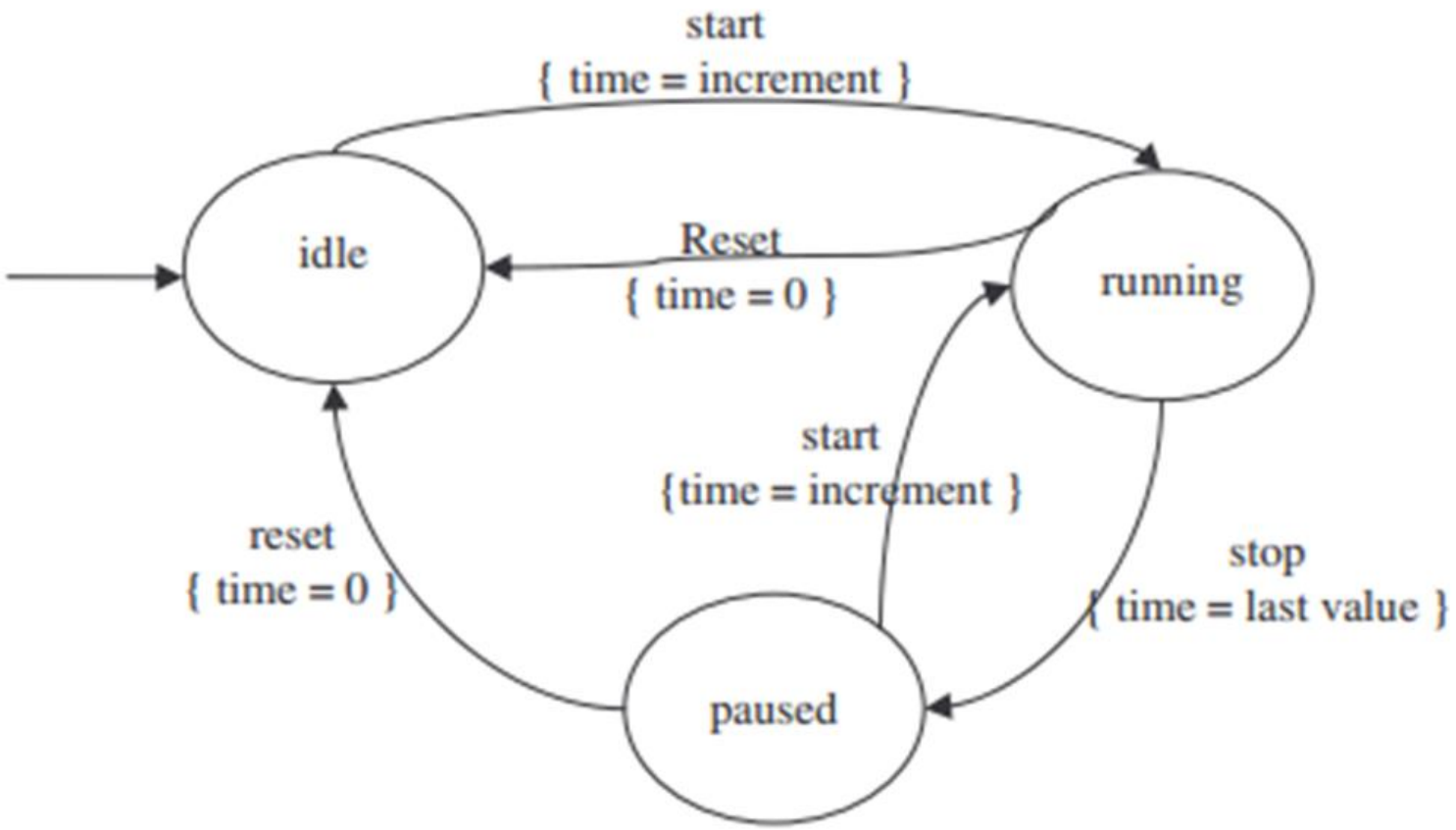
STD - Case - Soda vending

- Create the <u>state transition table</u>



STD - Case - Stopwatch

- Create the <u>state transition table</u>



Questions & answers